

Server Actions

use server · useActionState · Optimistic Updates · Security · vs API Routes

Akshay Chavhan | Target: Syngenta India + All Companies | April 2026

■ BEGINNER LEVEL

The Problem Before Server Actions

Every form submission or data mutation needed THREE separate files and lots of boilerplate. Server Actions collapsed this into a **single async function**.

BEFORE — API Routes (3 files)

```
1. Client Component
  const [loading, setLoading]
    = useState(false)

2. fetch call
  await fetch('/api/product',
    { method: 'POST', ... })

3. app/api/products/route.ts
  export async function POST(
    req: Request) {
    await db.product.create()
```

AFTER — Server Actions (1 function)

```
async function createProduct(
  formData: FormData) {
  'use server';
  await db.product.create({
    data: {
      name: formData.get('name')
    }
  });
  revalidatePath('/products');
}
```

What is a Server Action?

A Server Action is an **async function marked with "use server"** that runs on the Node.js server but can be triggered from Client Components or HTML forms. Think of it as a remote control — you press a button on the client, code runs on the server.

■ Key Point: Server Actions are for **MUTATIONS** (create, update, delete). Data fetching still uses async Server Components or Route Handlers.

Two Ways to Declare a Server Action

```
// WAY 1: 'use server' at TOP OF FILE
// Every exported async function in this file = Server Action
'use server';

export async function createProduct(formData: FormData) {
  await db.product.create({ data: { name: formData.get('name') } });
}

export async function deleteProduct(id: string) {
  await db.product.delete({ where: { id } });
}

// WAY 2: 'use server' INSIDE a function (inline)
// Only that specific function = Server Action
export default function ProductsPage() {

  async function createProduct(formData: FormData) {
    'use server'; // ← only this function is a Server Action
    await db.product.create({ data: { name: formData.get('name') } });
  }

  return (
    <form action={createProduct}>
      <input name='name' />
      <button type="submit">Add</button>
    </form>
  );
}
```

Most Basic Usage — Form + Server Action

```
// app/products/page.tsx — Server Component
import { db } from '@/lib/db';
import { revalidatePath } from 'next/cache';

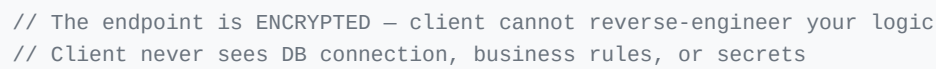
async function createProduct(formData: FormData) {
  'use server';
  const name = formData.get('name') as string;
  await db.product.create({ data: { name } });
  revalidatePath('/products'); // refresh page after mutation
}

export default function AddProductPage() {
  return (
    <form action={createProduct}>
      <input type='text' name='name' placeholder='Product name' />
      <button type="submit">Add Product</button>
    </form>
  );
}

// No useState. No fetch(). No API route. Just a function. ■
```

■ INTERMEDIATE LEVEL

How Server Actions Work Internally



useActionState (React 19+) connects a Server Action to a form and gives you loading state, error state, and result — completely replacing manual `useState` management.

```

// app/actions/products.ts
'use server';
import { revalidatePath } from 'next/cache';

type State = { error: string | null; success: boolean };

export async function createProduct(
  prevState: State,    // ← previous state from hook
  formData: FormData
): Promise<State> {
  const name = formData.get('name') as string;
  if (!name) return { error: 'Name is required', success: false };
  try {
    await db.product.create({ data: { name } });
    revalidatePath('/products');
    return { error: null, success: true };
  } catch {
    return { error: 'Failed to create product', success: false };
  }
}

// components/AddProductForm.tsx
'use client';
import { useActionState } from 'react';
import { createProduct } from '@app/actions/products';

export default function AddProductForm() {
  const [state, formAction, isPending] = useActionState(
    createProduct,
    { error: null, success: false } // initial state
  );

  return (
    <form action={formAction}>
      <input name='name' placeholder='Product name' />
      <button type="submit" disabled={isPending}>
        {isPending ? 'Adding...' : 'Add Product'}
      </button>
      {state.error && <p style={{color:'red'}}>{state.error}</p>}
      {state.success && <p style={{color:'green'}}>Added!</p>}
    </form>
  );
}

```

Calling Server Actions from Client Components (Without Form)

```
// Call Server Action directly like any async function
'use client';
import { deleteProduct } from '@app/actions/products';
import { useState } from 'react';

export default function DeleteButton({ productId }: { productId: string }) {
  const [isDeleting, setIsDeleting] = useState(false);

  const handleDelete = async () => {
    setIsDeleting(true);
    await deleteProduct(productId); // ← Server Action called directly
    setIsDeleting(false);
  };

  return (
    <button onClick={handleDelete} disabled={isDeleting}>
      {isDeleting ? 'Deleting...' : 'Delete'}
    </button>
  );
}
```

Validation with Zod — Full Production Pattern

```
'use server';
import { z } from 'zod';
import { revalidatePath } from 'next/cache';
import { redirect } from 'next/navigation';

const ProductSchema = z.object({
  name: z.string().min(1).max(100),
  price: z.number().positive(),
  category: z.enum(['seeds', 'pesticides', 'fertilizers']),
});

export async function createProduct(formData: FormData) {
  // Step 1: Extract raw data
  const raw = { name: formData.get('name'),
    price: Number(formData.get('price')),
    category: formData.get('category') };

  // Step 2: Validate
  const validated = ProductSchema.safeParse(raw);
  if (!validated.success) {
    return { error: validated.error.flatten().fieldErrors };
  }

  // Step 3: Save to DB
  await db.product.create({ data: validated.data });

  // Step 4: Clear cache
  revalidatePath('/products');
  revalidateTag('products');
}

// redirect('/products'); ← call OUTSIDE try/catch if needed
```

Optimistic Updates with useOptimistic

Show the result **instantly in UI** before server confirms. If server fails, React automatically reverts to the original state.

```
'use client';
import { useOptimistic } from 'react';
import { toggleLike } from '@app/actions/likes';

type Post = { id: string; liked: boolean; likes: number };

export default function LikeButton({ post }: { post: Post }) {
  const [optimisticPost, addOptimistic] = useOptimistic(
    post,
    (current, liked: boolean) => ({
      ...current,
      liked,
      likes: liked ? current.likes + 1 : current.likes - 1
    })
  );

  const handleLike = async () => {
    addOptimistic(!optimisticPost.liked); // Step 1: UI updates instantly
    await toggleLike(post.id);           // Step 2: Server call
    // If server fails: React reverts automatically ■
  };

  return (
    <button onClick={handleLike}>
      {optimisticPost.liked ? '♥■' : '■'} {optimisticPost.likes}
    </button>
  );
}
```

■ EXPERT LEVEL

Server Actions vs API Routes — When to Use Which

Feature	Server Actions	API Routes (route.ts)
Best for	Mutations from your own UI	Public endpoints, mobile, webhooks
Called by	Your Next.js app only	Anyone with the URL
Auth	Server-side, secure	Must check manually every time
Type safety	End-to-end automatic	Manual or Zod needed
Boilerplate	Very low — 1 function	Medium — full handler
Works without JS	Yes — native HTML form	No
File upload	Yes — FormData	Yes
Streaming response	No	Yes
CSRF protection	Automatic	Must implement manually

■ Simple Rule: Mutations from your own Next.js UI = Server Action. Public API endpoint for mobile apps, external services, or webhooks = API Route.

Progressive Enhancement — Works Without JavaScript

When you pass a Server Action to a native HTML form action attribute, the form works even when JavaScript is completely disabled. This is called **Progressive Enhancement**.

```
export default function ContactPage() {
  async function sendMessage(formData: FormData) {
    'use server';
    await db.message.create({ data: { body: formData.get('message') } });
    redirect('/thank-you');
  }

  return (
    <form action={sendMessage}>
      <textarea name="message" />
      <button type="submit">Send</button>
    </form>
  );
}

// With JS DISABLED:
//   Browser submits as native HTML form POST → Server Action runs ■
// With JS ENABLED:
//   Next.js intercepts → fetch POST → no page reload ■
// Same code. Both work.
```

Passing Extra Data with .bind()

When a form needs to know an ID (like which product to update), use **.bind()** to pre-attach arguments to the Server Action.

```

// app/products/page.tsx
import { updateProduct } from '@app/actions/products';

export default async function ProductsPage() {
  const products = await db.product.findMany();

  return (
    <ul>
      {products.map(product => {
        // Bind product.id as first argument
        const updateWithId = updateProduct.bind(null, product.id);
        return (
          <li key={product.id}>
            <form action={updateWithId}>
              <input name="name" defaultValue={product.name} />
              <button type="submit">Update</button>
            </form>
          </li>
        );
      })}
    </ul>
  );
}

// app/actions/products.ts
'use server';
export async function updateProduct(
  id: string, // ← comes from .bind()
  formData: FormData // ← comes from form submission
) {
  const name = formData.get('name') as string;
  await db.product.update({ where: { id }, data: { name } });
  revalidatePath('/products');
}

```

Security — Always Validate + Authenticate


```
'use server';
import { auth } from '@lib/auth';

export async function deleteProduct(id: string) {
  // 1. AUTHENTICATION – is user logged in?
  const session = await auth();
  if (!session) throw new Error('Unauthorized');

  // 2. AUTHORIZATION – does user OWN this product?
  const product = await db.product.findUnique({ where: { id } });
  if (product?.userId !== session.user.id) {
    throw new Error('Forbidden – not your product');
  }

  // 3. VALIDATION – is the input valid?
  if (!id || typeof id !== 'string') {
    throw new Error('Invalid ID');
  }

  // 4. MUTATION – safe to proceed
  await db.product.delete({ where: { id } });
  revalidatePath('/products');
}

// Next.js handles: CSRF protection (automatic)
// You handle:      Auth, Authorization, Validation
```

■ Critical Gotchas

■ Gotcha 1: `redirect()` must be OUTSIDE `try/catch`. `redirect()` works by throwing a special internal error. If you put it inside `try/catch`, your catch block catches the redirect and treats it as a failure. Put `redirect()` after the `try/catch` block.

■ Gotcha 2: Server Actions are POST only. They cannot be used for data fetching (GET). Use async Server Components for fetching data. Use Server Actions for mutations (create, update, delete) only.

■ Gotcha 3: Only serializable data can be passed as arguments to Server Actions. No functions, no class instances, no Date objects. Pass strings, numbers, FormData, and plain objects only.

■ Gotcha 4: `'use server'` must be in an ASYNC function. Putting it in a non-async function = build error. Every Server Action must be async even if it does not use `await`.

■ Best Practice: Put all Server Actions in a dedicated `actions/` folder (e.g. `app/actions/products.ts`). This keeps mutations organized, makes them reusable across components, and makes it easy to add auth checks in one place.

■ Best Practice: Always call `revalidatePath()` or `revalidateTag()` after a mutation. Without it, the user sees stale cached data after their action. Every create/update/delete should clear the relevant cache.

■ INTERVIEW Q&A WITH DETAILED ANSWERS

Beginner Level

BEGINNER**Q1. What is a Server Action in Next.js?**

Answer: A Server Action is an async function marked with "use server" that runs on the server but can be triggered from forms or Client Components. It eliminates the need for separate API routes for mutations. You can directly access the DB, validate data, and revalidate cache — all from one function the client triggers.

BEGINNER**Q2. What was the problem before Server Actions existed?**

Answer: Every mutation required three things: a Client Component with useState for loading/error state, a fetch() call to an API endpoint, and a separate app/api/route.ts file. Three files and lots of boilerplate for every form or button click. Server Actions collapse all of this into one async function.

BEGINNER**Q3. What is the difference between "use server" at file top vs inside a function?**

Answer: "use server" at the top of a file makes EVERY exported async function in that file a Server Action. Use this for dedicated action files. "use server" inside a single async function makes only THAT function a Server Action. Use this for inline actions defined inside Server Component pages.

Intermediate Level

INTERMEDIATE**Q4. What is useActionState and why use it instead of useState?**

Answer: useActionState (React 19+) is a hook that connects a Server Action to a form. It returns three things: the current state (error/success/data from the action), the form action function, and isPending boolean for loading state. Without it you need multiple useState hooks plus manual loading management. useActionState replaces all that boilerplate with one hook.

INTERMEDIATE**Q5. Can you call a Server Action without a form?**

Answer: Yes. Server Actions are just async functions. You can call them directly from any Client Component event handler — onClick, onChange, custom events. Import the action, await it in an async handler function, and call it with any arguments. The function still runs on the server even when called this way.

INTERMEDIATE**Q6. What is optimistic update and how does useOptimistic help?**

Answer: An optimistic update means updating the UI instantly before the server confirms the change. useOptimistic takes the current state and an updater function — you call addOptimistic() with the expected new state, UI updates immediately. Meanwhile the Server Action runs. If the server fails, React automatically reverts to the original state. This makes the app feel instant even with slow network.

INTERMEDIATE**Q7. What should you always call after a mutation in a Server Action?**

Answer: `revalidatePath()` or `revalidateTag()` — or both. Without them, the page still shows cached stale data after the mutation. `revalidatePath("/products")` clears the cache for that specific page. `revalidateTag("products")` clears all fetches tagged "products" across all pages. You can also call `redirect()` to send the user to a different page after success.

Expert / Syngenta-Style Questions

EXPERT**Q8. What is the difference between Server Actions and API Routes? When to use each?**

Answer: Server Actions: for mutations from your own UI, one function no API file, automatic CSRF, works without JS, type-safe end-to-end, called only by your app. API Routes: for public endpoints consumed by mobile apps, external services, or webhooks, can stream responses, called by anyone with the URL. Rule: mutations from your own Next.js UI = Server Action. Public endpoint = API Route.

EXPERT**Q9. Why must `redirect()` be placed outside `try/catch` in a Server Action?**

Answer: Next.js `redirect()` works internally by throwing a special error that Next.js intercepts to perform the redirect. If `redirect()` is inside a `try/catch` block, your catch clause catches this internal error and treats it as a regular failure — the redirect never happens. Always call `redirect()` after the `try/catch` block once you have confirmed the operation succeeded.

SYNGENTA**Q10. Design the complete Server Action flow for a Syngenta admin updating crop prices.**

Answer: In `app/actions/crops.ts` with "use server": (1) Auth check — `await auth()`, throw if no session or not admin role. (2) Validate input — Zod schema, price must be positive number, `cropId` must be valid string. (3) DB mutation — `db.crop.update()`. (4) Cache clear — `revalidateTag("crop-prices")` for all pages, `revalidatePath("/admin/crops")`. In the Client Component: `useActionState` for loading/error/success state, `useOptimistic` to update price in table instantly before server confirms.

EXPERT**Q11. What security checks are YOU responsible for in a Server Action?**

Answer: Next.js handles CSRF protection automatically. You are responsible for: (1) Authentication — check session exists with `auth()`. Throw or return error if not logged in. (2) Authorization — verify user is allowed to mutate THIS specific resource (not just any resource). (3) Input validation — validate and sanitize all `FormData` with Zod. Never trust client input directly. Skipping any of these creates security vulnerabilities even though the Action runs server-side.